

Documento de Requerimientos del Sistema: API de Gestión de Neumáticos (GesNeu) - v2.1 (Detallado)

1. Introducción

1.1 Propósito del Documento

Este documento describe los requerimientos funcionales y no funcionales para la API (Interfaz de Programación de Aplicaciones) de Gestión de Neumáticos (GesNeu). El objetivo principal es facilitar el seguimiento integral, la asignación eficiente, el control exhaustivo de la vida útil y la gestión detallada del ciclo de vida completo de los neumáticos utilizados en la flota de vehículos de carga de la empresa. Esto incluye desde la adquisición hasta el desecho, pasando por todos los eventos operativos y de mantenimiento.

1.2 Alcance del Proyecto

El proyecto abarca el diseño, desarrollo, prueba e implementación de una API RESTful robusta y escalable. Se utilizará Python con el framework FastAPI y SQLAlchemy para la interacción con una base de datos PostgreSQL. La API servirá como el backend central para todas las operaciones de gestión de neumáticos, permitiendo la integración con sistemas actuales o futuros de la empresa (ERP, sistemas de taller, telemática/IoT) y facilitando la toma de decisiones basada en datos. Se priorizará una arquitectura modular, mantenible y bien documentada.

1.3 Audiencia

Este documento está destinado a:

- **Equipo de Desarrollo:** Ingenieros de software, desarrolladores backend y frontend que construirán y mantendrán la API y las aplicaciones cliente.
- **Administradores de Bases de Datos:** Para entender el esquema de datos y los requisitos de integridad.
- **Equipo de Gestión de la Empresa:** Para comprender las capacidades del sistema, su alineación con los objetivos de negocio y para la toma de decisiones estratégicas.
- **Personal de Operaciones y Mantenimiento:** Futuros usuarios clave del sistema (Gestores de Flota, Jefes de Taller, Operadores de Mantenimiento) para validar que los requerimientos cubren sus necesidades operativas.
- **Equipo de QA/Pruebas:** Para la elaboración de planes y casos de prueba.
- **Otros Stakeholders:** Cualquier otra parte involucrada en el ciclo de vida del proyecto.

2. Descripción General

2.1 Contexto del Negocio y Problema

La empresa de servicios de carga opera una flota de vehículos pesados donde los neumáticos representan un costo operativo significativo y un factor crítico para la seguridad. La gestión actual, posiblemente manual o basada en sistemas fragmentados, presenta desafíos como:

- Dificultad para rastrear el historial completo y el rendimiento individual de cada neumático.
- Ineficiencia en la gestión de inventarios de neumáticos nuevos, usados, en reparación o para reencauche.
- Falta de datos consolidados para análisis de costos (Costo por Kilómetro - Cpk), comparación de marcas/modelos y optimización de políticas de reencauche/compra.
- Riesgos de seguridad por un seguimiento inadecuado de la profundidad del dibujo, presión, o asignación incorrecta de neumáticos reencauchados.
- Procesos manuales propensos a errores y lentos para el registro de eventos (montajes, desmontajes, inspecciones, etc.).

2.2 Objetivos del Sistema (Actualizado y Refinado)

Los objetivos del sistema API GesNeu, refinados tras la implementación inicial y depuración, son:

1. Optimizar Costos Operativos:

- Calcular y analizar el Costo por Kilómetro (Cpk) de forma precisa para cada neumático a lo largo de su ciclo de vida completo (original y reencauches).
- Facilitar la comparación de rendimiento y costo entre diferentes marcas, modelos y proveedores de servicios (reencauche, reparación).
- Gestionar eficientemente los ciclos de reencauche, identificando el momento óptimo y controlando el número máximo de reencauches por carcasa.
- Minimizar los desechos prematuros mediante un mejor seguimiento del desgaste y las causas de fallo.

2. Aumentar la Seguridad de la Flota:

- Garantizar el cumplimiento estricto de las políticas de uso diferenciadas para neumáticos nuevos vs. reencauchados/reparados en cada posición vehicular.
- Implementar alertas automáticas para profundidades de dibujo mínimas, presiones anómalas (requiere integración IoT o registro manual frecuente) y otros indicadores de riesgo.
- Asegurar la trazabilidad de cada neumático y su historial de mantenimiento.

3. Maximizar Rendimiento Kilométrico y Vida Útil:

- Realizar un seguimiento preciso del kilometraje acumulado por cada neumático en cada una de sus vidas (original y post-reencauche).
- Registrar y analizar el desgaste (profundidad del dibujo) a lo largo del tiempo y uso.
- Gestionar de forma informada el proceso de reencauche y reparación, seleccionando los proveedores adecuados y rastreando los resultados.
- Identificar patrones y causas comunes de desgaste o fallo prematuro.

4. Agilizar y Fiabilizar Procesos Operativos:

- Automatizar el seguimiento de neumáticos por ubicación (almacén, vehículo/posición) y estado (nuevo, en stock, instalado, en reparación, etc.).
- Proveer una interfaz API clara y eficiente para el registro de todos los eventos del ciclo de vida del neumático.
- Reducir la entrada manual de datos y la duplicidad de información.

5. Proveer Información Consolidada y Análisis Confiables:

- Generar reportes y KPIs para la toma de decisiones estratégicas sobre:
- Políticas de compra vs. reencauche.
- Selección de marcas/modelos de neumáticos.
- Evaluación de proveedores de neumáticos y servicios.
- Efectividad de las políticas de mantenimiento y rotación.

6. Centralizar Ciclo de Vida Completo del Neumático:

- Establecer una única fuente de verdad para toda la información relacionada con cada neumático, desde su adquisición (o registro inicial si ya existe) hasta su desecho final.
- Incluir todos los eventos relevantes: compra, instalación, desmontaje, inspecciones, rotaciones, envíos y recepciones de reparación/reencauche, ajustes de inventario, y desecho.

7. Facilitar la Integración:

- Proveer una API RESTful bien documentada (OpenAPI/Swagger) para permitir la integración con otros sistemas empresariales (ERP, TMS, sistemas de taller) y futuras tecnologías (sensores IoT).

8. Mejorar la Mantenibilidad y Escalabilidad de la API:

- Implementar una arquitectura de software clara y modular (ver Sección 3).
- Utilizar prácticas de codificación estandarizadas, incluyendo el uso de SQLAlchemy para la definición de modelos y schemas, y FastAPI para la creación de endpoints.
- Asegurar una cobertura de pruebas adecuada.

2.3 Características del Usuario y Roles del Sistema

Los usuarios interactuarán con la API (directamente o a través de aplicaciones cliente) según los roles asignados. Se definen los siguientes roles principales:

- **ADMIN (Administrador del Sistema):**

- Responsable de la configuración general del sistema.
- Gestión de usuarios y sus roles.
- Supervisión general de la integridad de los datos y logs del sistema.
- Puede tener permisos para realizar todas las operaciones si es necesario, pero su función principal no es la operativa diaria de neumáticos.

- **GESTOR (Gestor de Flota/Inventario):**

- Responsable de la supervisión del inventario de neumáticos.
- Toma de decisiones sobre compras, envíos a reencauche/reparación, y desecho de neumáticos.
- Análisis de rendimiento, costos (Cpk) y KPIs.
- Gestión de catálogos base (modelos, proveedores, parámetros).
- Puede realizar la mayoría de los eventos de neumáticos, especialmente aquellos que implican decisiones de gestión.

- **OPERADOR (Operador de Mantenimiento/Taller):**

- Responsable del registro de eventos operativos diarios.
- Ejecuta tareas como instalación, desmontaje, inspección, rotación de neumáticos.
- Puede registrar la entrada y salida de neumáticos de servicios de reparación/reencauche.
- Consulta el estado y la ubicación de los neumáticos.

- **(Potencial) OTROS_USUARIOS (Ej. Compras, Conductores):**

- Roles con permisos más limitados para consultas específicas o tareas muy acotadas, a definir según necesidad.

Se asume que todos los usuarios tendrán las credenciales necesarias para autenticarse en el sistema y que sus acciones estarán restringidas por su rol asignado.

3. Arquitectura y Estructura del Proyecto (Recomendaciones Clave)

Para asegurar la mantenibilidad, escalabilidad y claridad del código, la API GesNeu se desarrollará siguiendo una arquitectura por capas bien definida:

- **3.1 Configuración Centralizada (core/config.py):**

- Este módulo será el único responsable de cargar y proveer la configuración de la aplicación.
- Utilizará pydantic-settings para cargar variables desde un archivo .env (para desarrollo local) y desde variables de entorno del sistema (para producción).
- Incluirá:
 - SECRET_KEY, ALGORITHM, ACCESS_TOKEN_EXPIRE_MINUTES para JWT.
 - DATABASE_URL para la conexión a PostgreSQL.
 - API_V1_STR para el prefijo de las rutas de la API.
 - Umbrales de negocio (ej. UMBRAL_PROFUNDIDAD_MINIMA_MM).
 - Orígenes CORS permitidos (BACKEND_CORS_ORIGINS).

- **3.2 Gestión de Base de Datos:**

- database.py:
 - Contendrá la lógica para crear el motor de base de datos asíncrono (create_async_engine) usando la DATABASE_URL de la configuración.
 - Definirá el async_session_maker para crear sesiones de base de datos.
 - Incluirá la función init_db() para crear todas las tablas definidas por los modelos SQLAlchemy (usado principalmente para desarrollo y pruebas iniciales).
- core/dependencies.py:
 - Definirá la dependencia get_session() -> AsyncGenerator[AsyncSession, None] que será usada por todos los endpoints y servicios para obtener una sesión de base de datos. Esta función utilizará el async_session_maker de database.py.
 - También definirá oauth2_scheme = OAuth2PasswordBearer(...) para la autenticación.

- **3.3 Separación de Responsabilidades (Capas Lógicas):**

- **Modelos (models/):**

- Definirán la estructura de las tablas de la base de datos utilizando SQLAlchemy con table=True.
- Incluirán todos los campos, tipos de datos de columna SQLAlchemy (Column), claves primarias (primary_key=True), claves foráneas (foreign_key="tabla.campo"), índices (index=True), y constraints (unique=True, nullable=False).
- Definirán las relaciones entre tablas usando Relationship de SQLAlchemy, especificando back_populates para relaciones bidireccionales.
- Ejemplos: usuario.py, neumatico.py, evento_neumatico.py, vehiculo.py, etc.
- Se podrá crear un models/common.py para elementos comunes a varios modelos, como clases base de auditoría (TimestampModel) o tipos de columna reutilizables (TimestampTZ).

- **Schemas (schemas/):**

- Definirán la forma de los datos para las peticiones (entrada) y respuestas (salida) de la API, usando `SQLModel` (sin `table=True`) o `pydantic.BaseModel`.
- Incluirán validaciones de datos a nivel de API.
- Se recomienda una estructura común para cada entidad principal:
- **SchemaBase: Campos comunes.**
- **SchemaCreate(SchemaBase):** Campos requeridos para la creación, incluyendo aquellos que no se devuelven (ej. `password`).
- **SchemaUpdate(SQLModel):** Campos opcionales para la actualización.
- **SchemaRead(SchemaBase):** Campos que se devuelven en las respuestas, incluyendo IDs y campos de auditoría. Puede incluir schemas anidados para relaciones.
- `schemas/common.py`: Contendrá definiciones de Enums (`EstadoNeumaticoEnum`, `TipoEventoNeumaticoEnum`, etc.) y schemas genéricos como `Mensaje` o `PaginatedResponse`.
- Las importaciones entre schemas deben ser relativas (ej. `from .common import ...`) si están en el mismo paquete `schemas`.
- **CRUD (crud/):**
- Contendrán funciones que realizan las operaciones directas y reutilizables de base de datos (Crear, Leer, Actualizar, Eliminar) para cada modelo.
- Serán funciones asíncronas que aceptan una `AsyncSession` y los datos necesarios (usualmente un schema `Create` o `Update`, o IDs para leer/eliminar).
- Ejemplos: `crud_user.py` (con `get_by_username`, `create_user`), `crud_neumatico.py`.
- Esta capa es llamada únicamente por la capa de Servicios. No debe ser llamada directamente desde los Routers.
- **Servicios (services/):**
- Contendrán la lógica de negocio principal y la orquestación de operaciones.
- Serán clases (ej. `NeumaticoService`, `AlertService`) que reciben una `AsyncSession` en su constructor (inyectada).
- Llamarán a las funciones CRUD para interactuar con la base de datos.
- Implementarán reglas de negocio complejas, validaciones que involucren múltiples entidades o estados.
- Pueden llamar a otros servicios si es necesario.
- Manejarán la lógica específica de cada tipo de evento de neumático.
- Lanzarán excepciones personalizadas (ej. `InvalidOperationError`, `InsufficientDataError`) que luego pueden ser manejadas en los routers.
- **Routers (routers/):**
- Definirán los endpoints de la API usando `APIRouter` de FastAPI.

- Usarán Depends para inyectar la sesión de BD (get_session), el usuario actual (get_current_active_user), y los servicios necesarios.
- Validarán los datos de entrada usando los schemas Create o Update en los parámetros de las funciones de ruta.
- Llamarán a los métodos de la capa de Servicios para ejecutar la lógica de negocio.
- Formatearán las respuestas usando los schemas Read.
- Manejarán las excepciones lanzadas por los servicios y las convertirán en respuestas HTTP apropiadas (ej. HTTPException).
- **3.4 Manejo de Importaciones:**
- Se debe prestar especial atención a las rutas de importación para evitar ImportError y errores de "módulo no encontrado".
- Para resolver importaciones circulares (ej. entre models y schemas si un modelo necesita un tipo de un schema y viceversa), la estrategia principal es mover las definiciones que causan el ciclo (especialmente Enums o tipos base simples) a un módulo más fundamental y neutral, como schemas/common.py.
- Ambos módulos (models/neumatico.py y schemas/neumatico.py, por ejemplo) importarían el Enum EstadoNeumaticoEnum desde schemas.common.py.
- Evitar la redefinición de la misma clase o Enum en múltiples archivos.

4. Requerimientos Funcionales (Detallados)

4.1 Gestión de Catálogos Base

- **RF30-RF34: Tipos de Vehículo** (Mantener como en el original, asegurando que TipoVehiculoCreate y TipoVehiculoUpdate se usen para la entrada y TipoVehiculoRead para la salida. La desactivación es soft delete).
- **RF35: Gestionar Fabricantes de Neumáticos** (CRUD completo para FabricanteNeumatico. Usar schemas FabricanteNeumaticoCreate, Read, Update. nombre y codigo_abreviado deben ser únicos. Soft delete).
- **RF36: Gestionar Modelos de Neumáticos** (CRUD completo para ModeloNeumatico. Usar schemas. Campos obligatorios: fabricante_id, nombre_modelo, medida, profundidad_original_mm. Otros campos como tipo_neumatico, diametro_aro_r, ancho_mm, perfil_mm, indice_carga, indice_velocidad, presion_recomendada_psi, permite_reencauche, reencauches_maximos, patron_dibujo, tipo_servicio deben ser validados. Soft delete).
- **RF37: Gestionar Proveedores** (CRUD completo para Proveedor. Usar schemas. tipo debe usar TipoProveedorEnum. rfc y nombre podrían ser únicos. Soft delete).
- **RF38: Gestionar Motivos de Desecho** (CRUD completo para MotivoDesecho. Usar schemas. codigo debe ser único. Soft delete).

- **RF57: Gestionar Almacenes/Ubicaciones Físicas** (CRUD completo para Almacen. Usar schemas. codigo y nombre deben ser únicos. Soft delete. Validar si se puede desactivar un almacén con neumáticos).
- **RF58: Gestionar Parámetros de Inventario** (CRUD completo para ParametroInventario. Usar schemas. tipo_parametro usa TipoParametroEnum. La combinación (tipo_parametro, modelo_id, almacen_id) debe ser única. almacen_id es opcional para parámetros generales del modelo. Estos parámetros (ej. PROFUNDIDAD_MINIMA) serán usados por AlertService).

4.2 Configuración de Vehículos y Posiciones

- **RF21: Definir Configuración de Ejes por Tipo de Vehículo** (ConfiguracionEje): CRUD completo. Asociado a TipoVehiculo. Incluye: numero_eje, nombre_eje, tipo_eje (TipoEjeEnum), numero_posiciones, neumaticos_por_posicion, posiciones_duales (bool), permite_reencauchados (bool).
- **RF23: Definir Posiciones de Montaje por Eje** (PosicionNeumatico): CRUD completo. Asociado a ConfiguracionEje. Incluye: codigo_posicion (único dentro del tipo de vehículo/configuración de eje), lado (LadoVehiculoEnum), posicion_relativa (ej. 1, 2 para duales), es_interna (bool), es_direccion (bool), es_traccion (bool).
- **RF39: Validación de Posición en Asignación:** Al registrar un evento de INSTALACION, el NeumaticoService debe validar que la posicion_montaje_id sea compatible con el vehiculo_id (a través de su tipo_vehiculo_id y las ConfiguracionEje y PosicionNeumatico asociadas).

4.3 Gestión de Neumáticos (Ciclo de Vida Individual)

- **RF01 (Reemplazado por Evento de Compra):** La creación de neumáticos se maneja a través del evento COMPRA (ver RF_EVENTO_01).
- **RF02: Consultar Neumático:**
 - Endpoint GET /neumaticos/{neumatico_id}.
 - Debe devolver NeumaticoRead que incluya datos del neumático, su modelo (marca, medida, etc.), estado actual, ubicación actual (almacén o vehículo/posición con detalles), kilometraje_acumulado, vida_actual, reencauches_realizados, fecha_compra, costo_compra.
 - Opcionalmente, podría incluir un resumen de las últimas alertas activas o información del gemelo si está instalado en una posición dual.
- **RF03: Listar Neumáticos:**
 - Endpoint GET /neumaticos/.
 - Debe permitir paginación (skip, limit).
 - Debe permitir filtros por: estado_actual, modelo_id, marca_id, medida, ubicacion_almacen_id, ubicacion_actual_vehiculo_id, es_reencauchado, rango de fecha_compra, etc.
 - Debe devolver una lista de NeumaticoRead.

- **RF04: Actualizar Neumático:**

- Endpoint PUT /neumaticos/{neumatico_id}.
- Debe usar un schema NeumaticoUpdate que permita modificar campos que no son gestionados por eventos (ej. corrección de numero_serie si no hay eventos asociados, dot si se ingresó mal inicialmente, notas generales del neumático).
- El NeumaticoService debe validar qué campos se pueden actualizar y en qué condiciones.
- **RF05 (Reemplazado por Evento de Desecho):** El desecho se maneja a través del evento DESECHO (ver RF_EVENTO_01).

4.4 Gestión de Vehículos

- (Mantener RF06, RF07, RF08, RF09, RF40 como en el documento original. Asegurar que los endpoints usen schemas Create, Read, Update apropiados y que la desactivación sea un soft delete).

4.5 Gestión de Eventos de Neumáticos (Centralizado y Estandarizado)

- **RF_EVENTO_01: Registrar Evento Genérico de Neumático:**

- Endpoint principal: POST /neumaticos/eventos/.
- **Schema de Entrada:** schemas.evento_neumatico.EventoNeumaticoCreate. Este schema debe ser la única estructura para todos los tipos de eventos. Contendrá todos los campos posibles, muchos de ellos opcionales.
- Campos clave obligatorios para la mayoría de los eventos: tipo_evento, fecha_evento.
- neumatico_id es obligatorio excepto para COMPRA.
- **Campos estandarizados (usar estos nombres en el payload y en el servicio):**
 - fecha_evento: datetime (UTC)
 - kilometraje_vehiculo: Optional[float]
 - profundidad_remanente: Optional[float]
 - presion_psi: Optional[float]
 - proveedor_id: Optional[uuid.UUID] (Para compra o proveedor de servicio externo)
 - almacen_destino_id: Optional[uuid.UUID]
 - posicion_montaje_id: Optional[uuid.UUID]
 - motivo_desecho_id: Optional[uuid.UUID]
 - estado_neumatico_resultante: Optional[EstadoNeumaticoEnum]
 - costo_evento: Optional[float] (costo del servicio, no del neumático)
- **Campos específicos para el evento de COMPRA (usados para crear el Neumatico):**
 - numero_serie: str
 - modelo_id: uuid.UUID
 - marca_id: uuid.UUID (puede ser inferido por el servicio desde modelo_id)
 - medida: str
 - dot: str
 - profundidad_inicial: float
 - fecha_compra: Optional[date] (si es diferente de fecha_evento)

- costo_compra: Optional[float]
- **Lógica de Servicio** (NeumaticoService.registrar_evento):
- Recibe EventoNeumaticoCreate y current_user.
- Valida que el current_user tenga permiso para el tipo_evento (usando check_permission_for_event_type de core.security).
- Llama a un método privado _handle_<tipo_evento>(evento_in, db_neumatico_opcional, current_user, fecha_evento_dt):
- _handle_compra: Valida campos de compra. Crea un nuevo Neumatico con los datos provistos en evento_in. Asigna el id del nuevo neumático a evento_in.neumatico_id.
- _handle_instalacion: Valida vehiculo_id, posicion_montaje_id, kilometraje_vehiculo. Actualiza Neumatico.estado_actual a INSTALADO, ubicacion_actual_vehiculo_id, ubicacion_actual_posicion_id, ubicacion_almacen_id (a None), km_instalacion, fecha_instalacion.
- _handle_desmontaje: Valida kilometraje_vehiculo. Calcula y actualiza Neumatico.kilometraje_acumulado. Actualiza Neumatico.estado_actual (a EN_STOCK o según evento_in.estado_neumatico_resultante), ubicacion_almacen_id (si aplica, requiere evento_in.almacen_destino_id), limpia campos de instalación.
- _handle_inspeccion: Registra profundidad_remanente, presion_psi. Llama a AlertService.check_and_create_alerts.
- _handle_rotacion: Actualiza Neumatico.ubicacion_actual_posicion_id (y ubicacion_actual_vehiculo_id si cambia de vehículo). Requiere posicion_montaje_id (destino) y kilometraje_vehiculo.
- _handle_reparacion_entrada: Cambia estado a EN_REPARACION. Requiere proveedor_id (taller).
- _handle_reparacion_salida: Cambia estado a EN_STOCK. Requiere almacen_destino_id.
- _handle_reencauche_entrada: Cambia estado a EN_REENCAUCHE. Requiere proveedor_id. Valida Neumatico.reencauches_realizados < ModeloNeumatico.reencauches_maximos.
- _handle_reencauche_salida: Cambia estado a EN_STOCK. Incrementa Neumatico.reencauches_realizados y vida_actual. Actualiza Neumatico.profundidad_inicial y profundidad_inicial_mm con evento_in.profundidad_remanente. Resetea Neumatico.kilometraje_acumulado a 0. Marca Neumatico.es_reencauchado = True. Requiere almacen_destino_id, profundidad_remanente.
- _handle_desecho: Cambia estado a DESECHADO. Requiere motivo_desecho_id. Actualiza Neumatico.fecha_desecho. Valida que no esté instalado.
- _handle_ajuste_inventario: Actualiza Neumatico.estado_actual y su ubicación según evento_in.estado_neumatico_resultante y almacen_destino_id o vehiculo_id/posicion_montaje_id.
- Después del _handle_<tipo_evento>, el NeumaticoService actualiza Neumatico.actualizado_en, actualizado_por, fecha_ultimo_evento.

- Crea el registro `EventoNeumatico` en la base de datos con los datos de `evento_in` (excluyendo campos que son solo para la creación del neumático en caso de COMPRA).
- Realiza `commit` de la transacción.
- Devuelve `Tuple[NeumaticoRead, EventoNeumaticoRead]`.
- **RF12: Consultar Neumáticos Instalados:**
- Endpoint `GET /neumaticos/instalados`.
- Debe devolver una lista de `schemas.neumatico.NeumaticoInstaladoItem`, que incluya `id` (neumático), `numero_serie`, `medida`, `vehiculo_placa`, `posicion_codigo`, `fecha_instalacion`, `kilometraje_instalacion`.
- El servicio realizará un `join` entre `Neumatico`, `Vehiculo` y `PosicionNeumatico`.
- **RF13: Consultar Historial de Eventos:**
- Endpoint `GET /neumaticos/{neumatico_id}/historial`.
- Devuelve `List[EventoNeumaticoRead]` para el `neumatico_id` dado, ordenado por `fecha_evento` descendente.

4.6 Gestión Operativa y Mantenimiento

- (La mayoría de estos RFs se cubren con el sistema de Eventos RF_EVENTO_01).
- **RF16: Restricción de Asignación de Reencauchados:** El `NeumaticoService._handle_instalacion` debe validar que si `Neumatico.es_reencauchado` es `True`, la `PosicionNeumatico` (a través de su `ConfiguracionEje`) tenga `permite_reencauchados = True`.
- **RF17: Indicador de Neumático Reencauchado:** El schema `NeumaticoRead` debe incluir `es_reencauchado` y `reencauches_realizados`.
- **RF63: Registrar Datos Estructurados Adicionales en Eventos:** El campo `notas: Optional[str]` en `EventoNeumaticoBase` puede usarse para texto libre. Para datos más estructurados, se podría considerar un campo `datos_adicionales: Optional[JSON]` en `EventoNeumaticoBase` y el modelo `EventoNeumatico`, o definir schemas específicos que hereden de `EventoNeumaticoCreate` para ciertos tipos de evento si la complejidad lo justifica. Por ahora, `notas` puede contener JSON si es necesario, pero sin validación estricta a nivel de schema base.

4.7 Control de Vida Útil y Desgaste

- **RF18: Registrar Mediciones en Eventos con Comparación:** Cubierto por `EventoNeumaticoCreate` (campos `kilometraje_vehiculo`, `profundidad_remanente`, `presion_psi`). La lógica de comparación con umbrales y valores recomendados reside en el `NeumaticoService` (especialmente en `_handle_inspeccion`) y en `AlertService`.
- **RF19: Calcular Kilometraje Recorrido por Ciclo:** El `NeumaticoService._handle_desmontaje` (y `_handle_reencauche_salida` para resetear) es responsable de actualizar `Neumatico.kilometraje_acumulado`.

- **RF20: Consultar Kilometraje Acumulado Total:** El campo `Neumatico.kilometraje_acumulado` en `NeumaticoRead` provee esto para la vida actual. Para el total de todas las vidas, se requeriría una lógica de consulta más compleja o un campo adicional en `Neumatico` que acumule el total histórico. (Considerar para Fase 2 si no es el `kilometraje_acumulado` actual).
- **RF26 y RF27: Seguimiento de Vida Útil y Desgaste Estimado vs. Real:** Estos son análisis que se pueden construir sobre los datos recolectados. La API debe proveer los datos base. Las alertas se manejan en RF28.

4.8 Gestión de Stock y Almacén

- **RF44: Visualizar Stock Actual por Ubicación:** El endpoint `GET /neumaticos/` con filtros adecuados (`estado_actual=EN_STOCK`, `ubicacion_almacen_id`) debe permitir esto.
- **RF45: Trazabilidad de Movimientos de Stock:** El historial de eventos (RF13) y los campos de ubicación en `Neumatico` proveen esta trazabilidad.
- **RF46: Alertas de Stock Mínimo:** `AlertService` debe verificar contra `ParametroInventario` de tipo `STOCK_MINIMO` después de eventos que modifiquen el stock en un almacén (ej. `COMPRA`, `DESMONTAJE a almacén`, `TRANSFERENCIA_UBICACION`).
- **RF47: Identificar Stock Antiguo/Obsoleto:** El endpoint `GET /neumaticos/` con filtros por `fecha_compra` o `fecha_ultimo_evento` puede ayudar a identificar esto.

4.9 Reportes y Análisis

- (Mantener RF25, RF48, RF49, RF50, RF61, RF74, RF82. La API debe proveer endpoints de consulta que faciliten la extracción de datos para estos reportes y cálculos. Algunos KPIs podrían ser endpoints dedicados).

4.10 Alertas y Automatización

- **RF28: Generar Alertas Operativas y de Mantenimiento (AlertService):**
- Debe ser invocado por `NeumaticoService` después de eventos como `INSPECCION` (para profundidad baja), o cuando se actualiza el stock (para `STOCK_MINIMO`).
- Tipos de alerta a considerar: `PROFUNDIDAD_BAJA`, `STOCK_MINIMO`, `LIMITE_REENCAUCHES_ALCANZADO`.
- **RF81: Gestión de Alertas Generadas:** Se necesita un CRUD para la entidad `Alerta` (`models/alerta.py`, `schemas/alerta.py`, `crud/crud_alerta.py`, `routers/alertas.py`). Permitir listar alertas (filtrar por activas, tipo, neumático, etc.) y actualizar su estado (ej. a `GESTIONADA`).

4.11 Utilidades de Datos y Operaciones Avanzadas

- (Mantener RF64, RF65, RF80, RF83 como en el documento original, marcándolos como Fase 2 si es necesario).

4.12 Futuras Capacidades (Fase 2 / Opcional)

- (Mantener RF66, RF67, RF71, RF72, RF73 como en el documento original).

5. Requerimientos No Funcionales

5.1 Rendimiento

- **RNF01: Tiempo de Respuesta:** La mayoría de las operaciones CRUD y de registro de eventos deben completarse en < 2 segundos bajo carga normal. Las consultas complejas para reportes pueden tener tiempos mayores, pero deben ser optimizadas.
- **RNF02: Escalabilidad:** La API debe ser capaz de manejar un volumen creciente de neumáticos, vehículos y eventos sin degradación significativa del rendimiento. Considerar paginación en todos los endpoints de listado.

5.2 Seguridad

- **RNF03: Autenticación:**
 - Se utilizarán JWT Bearer tokens.
 - El endpoint POST /auth/token generará los tokens.
 - Las funciones de creación/verificación de tokens y hash de contraseñas residirán en core/security.py.
- **RNF04: Autorización (RBAC):**
 - Roles de usuario definidos: ADMIN, GESTOR, OPERADOR (y otros si son necesarios).
 - La dependencia get_current_active_user de core/security.py se usará en los endpoints para obtener el usuario autenticado y su rol.
 - Para el endpoint de creación de eventos (POST /neumaticos/eventos/), la autorización se basará en el tipo_evento enviado en el payload. La función check_permission_for_event_type en core/security.py usará el diccionario ROLES_PERMITIDOS_POR_EVENTO para validar si el rol del usuario actual puede realizar dicho tipo de evento.
- **Definición de ROLES_PERMITIDOS_POR_EVENTO (ejemplo, ajustar según negocio):**
 - COMPRA: ["GESTOR", "ADMIN"]
 - INSTALACION: ["OPERADOR", "GESTOR", "ADMIN"]
 - DESMONTAJE: ["OPERADOR", "GESTOR", "ADMIN"]
 - INSPECCION: ["OPERADOR", "GESTOR", "ADMIN"]
 - ROTACION: ["OPERADOR", "GESTOR", "ADMIN"]
 - REPARACION_ENTRADA: ["OPERADOR", "GESTOR"] (ADMIN podría agregarse si es necesario)
 - REPARACION_SALIDA: ["OPERADOR", "GESTOR"]
 - REENCAUCHE_ENTRADA: ["GESTOR", "ADMIN"]
 - REENCAUCHE_SALIDA: ["OPERADOR", "GESTOR"]
 - DESECHO: ["GESTOR", "ADMIN"]
 - AJUSTE_INVENTARIO: ["GESTOR", "ADMIN"]

- (Asegurar que todos los TipoEventoNeumaticoEnum estén mapeados)
- Otros endpoints (CRUDs de catálogos, vehículos, etc.) también deben ser protegidos según el rol (ej. solo ADMIN o GESTOR pueden crear/modificar ciertos catálogos).
- **RNF05: Protección de Datos:** (Mantener como en el original - HTTPS, protección contra inyección SQL a través del ORM).

5.3 Usabilidad

- **RNF06: Facilidad de Uso (API):** API RESTful con endpoints intuitivos. Documentación OpenAPI (Swagger UI y ReDoc) generada automáticamente por FastAPI. Mensajes de error claros y útiles.

5.4 Fiabilidad

- **RNF07: Disponibilidad:** (Mantener como en el original).
- **RNF08: Integridad de Datos:** Asegurada mediante el uso de tipos de datos correctos, constraints NOT NULL en la BD, claves foráneas, y transacciones ACID para operaciones compuestas (ej. registro de evento y actualización de neumático deben ser atómicos).

5.5 Mantenibilidad

- **RNF09: Facilidad de Mantenimiento:** Código bien estructurado siguiendo la arquitectura de capas, código limpio y comentado, uso de type hints de Python, y un conjunto comprensivo de pruebas unitarias y de integración.

5.6 Retención de Datos

- **RNF10: Retención de Datos (Soft Delete):** Para entidades maestras críticas (Neumatico, Vehiculo, Usuario, Proveedor, ModeloNeumatico, FabricanteNeumatico, Almacen, TipoVehiculo, etc.), se implementará soft delete mediante un campo activo: bool y, opcionalmente, fecha_baja: Optional[datetime]. Los registros no se eliminarán físicamente de la base de datos. Los endpoints de listado por defecto solo mostrarán registros activos.

5.7 Consistencia de Unidades

- **RNF11: Consistencia en Unidades de Medida:** (Mantener como en el original).

6. Consideraciones Adicionales (Cálculos y Factores)

- (Mantener sección como en el original, estos son más conceptuales para la lógica de negocio que la API podría soportar o para análisis externos).

Anexo A: Definición de Enumeraciones Clave (Referencia Centralizada en schemas/common.py)

- EstadoNeumaticoEnum:
- NUEVO: Estado inicial al registrar sin compra directa (ej. inventario existente).
- EN_STOCK: Comprado y disponible en almacén.
- INSTALADO: Montado en un vehículo (sinónimo de EN_USO).

- EN_USO: Montado en un vehículo.
- EN_ALMACEN: Guardado en almacén (puede ser nuevo o usado, diferente de EN_STOCK si se quiere distinguir el origen).
- PARA_REPARACION: Designado para ser reparado.
- EN_REPARACION: Actualmente en proceso de reparación.
- REPARADO: Reparación completada, listo para almacén o uso.
- PARA_REENCAUCHE: Designado para ser reencauchado.
- EN_REENCAUCHE: Actualmente en proceso de reencauche.
- REENCAUCHADO: Reencauche completado, listo para almacén o uso.
- PARA_DESECHO: Designado para ser desecho.
- DESECHADO: Dado de baja permanentemente.
- VENDIDO: Neumático vendido.
- EN_TRANSITO: Moviéndose entre ubicaciones/almacenes.

(Revisar y consolidar esta lista para que sea definitiva y no redundante, ej. INSTALADO vs EN_USO, EN_STOCK vs EN_ALMACEN).

- TipoEventoNeumaticoEnum:
- COMPRA
- INSTALACION
- DESMONTAJE (general, puede ir a stock, reparación, desecho)
- ROTACION
- INSPECCION
- REPARACION_ENTRADA (envío a taller de reparación)
- REPARACION_SALIDA (retorno de taller de reparación)
- REENCAUCHE_ENTRADA (envío a reencauchadora)
- REENCAUCHE_SALIDA (retorno de reencauchadora)
- DESECHO (confirmación de desecho)
- AJUSTE_INVENTARIO (corrección manual de estado/ubicación)
- ASIGNACION_A_ALMACEN (movimiento explícito a un almacén)
- VENTA
- MOVIMIENTO_ENTRE_ALMACENES
- BAJA_POR_ROBO_EXTRAVIO
- TRANSFERENCIA_UBICACION (sinónimo de MOVIMIENTO_ENTRE_ALMACENES o ASIGNACION_A_ALMACEN, considerar unificar)
- DESMONTE_POR_FIN_VIDA_UTIL (subtipo de DESMONTAJE)
- DESMONTE_TEMPORAL (subtipo de DESMONTAJE)

(Revisar y consolidar esta lista para evitar redundancias y asegurar que cubra todos los casos de uso).

- LadoVehiculoEnum: IZQUIERDO, DERECHO, CENTRAL, INDETERMINADO.

- TipoEjeEnum: DIRECCION, TRACCION, LIBRE, PORTADOR, ARRASTRE.
- TipoProveedorEnum: FABRICANTE, DISTRIBUIDOR, SERVICIO_REPARACION, SERVICIO_REENCAUCHE, OTRO.
- TipoParametroEnum: STOCK_MINIMO, STOCK_MAXIMO, PROFUNDIDAD_MINIMA, PROFUNDIDAD_MINIMA_REENCAUCHE.